30 Medium-Level Java HashMap Questions

Output prediction • Concepts • Debugging • Coding practice

Section A: Output-Based Questions

Q1. What is the output?

```
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 10);
map.put("B", 20);
map.put("A", 30);

System.out.println(map.size());
System.out.println(map.get("A"));
```

Q2. What is the output?

```
HashMap<Integer, String> map = new HashMap<>();
map.put(1, "Java");
map.put(2, "Python");
map.put(3, "Java");

System.out.println(map.size());
System.out.println(map.containsValue("Java"));
```

Q3. Predict the output.

```
HashMap<String, Integer> map = new HashMap<>();
map.put(null, 100);
map.put("A", null);
map.put(null, 200);

System.out.println(map.size());
System.out.println(map.get(null));
System.out.println(map.get("A"));
```

Q4. What will be printed?

```
HashMap<Integer, String> map = new HashMap<>();
map.put(10, "Ten");
map.put(20, "Twenty");

System.out.println(map.get(30));
System.out.println(map.containsKey(30));
```

Q5. Find the output.

```
HashMap<String, Integer> map = new HashMap<>();
map.put("Java", 10);
map.put("Python", 20);

map.put("Java", map.get("Java") + 5);

System.out.println(map.get("Java"));
```

Q6. What is the output?

```
HashMap<Integer, Integer> map = new HashMap<>();
map.put(1, 10);
map.put(2, 20);
map.put(3, 30);

map.remove(2);

System.out.println(map.size());
System.out.println(map.containsKey(2));
```

Q7. Predict the result. What can you conclude about the order of keys?

```
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 1);
```

```
map.put("B", 2);
map.put("C", 3);

for (String key : map.keySet()) {
    System.out.print(key + " ");
}
```

Q8. What is the output?

```
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 10);

map.putIfAbsent("A", 50);
map.putIfAbsent("B", 20);

System.out.println(map);
```

Q9. Predict the output.

```
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 10);
map.put("B", 20);

map.replace("A", 10, 100);
map.replace("B", 50, 200);

System.out.println(map);
```

Q10. What will this print?

```
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 10);
map.put("B", 20);

map.compute("A", (key, value) -> value + 5);
map.computeIfAbsent("C", key -> 30);

System.out.println(map);
```

Section B: Conceptual Questions

Q11. Explain how HashMap stores key-value pairs internally.

Q12. What happens when two different keys have the same hash code?

Q13. Explain the difference between hashCode() and equals() in a HashMap.

Q14. Why should keys used in a HashMap ideally be immutable?

Q15. What is the difference between HashMap and Hashtable?

Q16. Explain the difference between put(), putIfAbsent(), and replace().

Q17. What happens if you insert a key whose hashCode() is the same as an existing key's hash code but equals() returns false?

Q18. Can a HashMap contain duplicate values? Can it contain duplicate keys? Explain.

Q19. What is the difference between get(key) and getOrDefault(key, defaultValue)?

Q20. What is the purpose of the load factor and initial capacity in a HashMap?

Section C: Debugging and Tricky Questions

Q21. Identify the problem, if any.

```
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 10);
map.put("B", 20);

if (map.get("C") == 0) {
    System.out.println("Found");
} else {
```

```
    System.out.println("Not Found");
}
```

Q22. What happens when this code runs? Explain why modifying a map while iterating over it can be problematic.

```
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 10);
map.put("B", 20);

for (String key : map.keySet()) {
    map.remove(key);
}
```

Q23. Find the bug in this code and explain how to fix it.

```
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 10);

int value = map.get("B");
System.out.println(value);
```

Q24. Predict the output.

```
HashMap<String, Integer> map = new HashMap<>();
map.put("A", null);

map.putIfAbsent("A", 100);

System.out.println(map.get("A"));
```

Q25. What is the problem with this key class? Discuss what can happen if equals() and hashCode() are implemented using the mutable id field.

```
class Student {
    int id;

    Student(int id) {
        this.id = id;
    }
}

HashMap<Student, String> map = new HashMap<>();
Student s = new Student(1);
map.put(s, "Kajal");

s.id = 2;

System.out.println(map.get(s));
```

Section D: Coding Questions

Q26. Frequency Counter: Write a Java program that counts how many times each number occurs in an array using a HashMap.

```
int[] arr = {1, 2, 1, 3, 2, 1};
// Expected frequencies:
// 1 -> 3
// 2 -> 2
// 3 -> 1
```

Q27. First Non-Repeating Character: Given a string, use a HashMap to find the first character that occurs exactly once.

```
Input:  "swiss"
Output: w
```

Q28. Two Sum: Given an integer array and a target, return the indices of two numbers whose sum equals the target. Use a HashMap for an efficient solution.

```
Input: nums = [2, 7, 11, 15], target = 9
Output: [0, 1]
```

Q29. Find Duplicate Elements: Write a program using HashMap to identify all elements that occur more than once in an array.

```
Input:  [4, 2, 4, 3, 2, 5]
Output: 4, 2
```

Q30. Student Marks Management: Create a HashMap to store student names and marks. Add five students, update a student's marks, find the highest scorer, display students scoring above 75, and check whether a particular student exists.

Practice tip: Solve Q1–Q10 without running the code, explain Q11–Q25 aloud, and implement Q26–Q30 in Java.